LAB 3: MQTT y ESP32

José Paca, *UTEC,* Daniel Casquino, *UTEC,* Nicolas Stigler, *UTEC,* Sofia Ku, *UTEC*

I. Introduction

THIS laboratory approaches the connection between data acquisition on an ESP32 and message distribution using MQTT. The guide organizes experiments in connectivity, telemetry, control and serial integration [1].

I-A. Theoretical Framework

MQTT uses the publisher–subscriber model: a client publishes to a topic, and the server distributes messages according to subscriptions. MQTT 3.1.1 requires an ordered, bidirectional transport, like TCP. Its QoS level 0 offers delivery at most once; it does not guarantee that every message will reach the receiver [2]. Therefore, simply observing a connection is not enough to quantify message loss or receipt of sensor readings.

In the Arduino-ESP32 API, `analogRead` returns a raw, uncalibrated ADC code, while `analogReadMilliVolts` provides a calibrated result in millivolts [3]. An ADC code obtained from a divider with an LDR does not directly equate to illuminance in lux. The relation with illumination requires characterizing the sensor, circuit, and reference conditions; this conversion is not performed here.

PubSubClient 2.8 selects MQTT 3.1.1 by default. Its overload `publish(topic,payload)` constructs a message without retention and without QoS bits 1/2; `loop()` handles client communication [4]. These properties describe the library and allow interpretation of the CP2 sketch; they do not replace a receiver capture.

I-B. Research Background and Positioning of This Work

Hunkeler, Truong, and Stanford-Clark proposed MQTT-S to extend publisher–subscriber communication to sensor networks with energy and bandwidth constraints [5]. The prior work shows the adaptation of the model to other transport mechanisms. The present lab uses MQTT over a WiFi/TCP connection; it does not implement MQTT-S or reproduce its evaluations. This brief review provides relevant background and a reference standard; it is not intended to be a comprehensive review of the state of the art.

II. Methodology

II-A. Componentes y configuración de CP2

La copia de código preparada en `checkpoint2/Checkpoint2_LDR/` usa ESP32, WiFi y PubSubClient; lee GPIO35, imprime a 115200 baudios y publica el valor crudo como texto en `lab/ldr`. El bucle contiene una espera de 1000 ms: no representa una medición de período exacto, pues las operaciones de red también consumen tiempo. La configuración local se mantiene separada del código versionado.
Modelo de placa, versiones de software, resistencia, orientación del divisor y alimentación: por documentar.

B1. Diagrama de bloques de CP2: por documentar.

Figura 1. Arquitectura de adquisición y distribución MQTT de CP2 (diagrama pendiente).

F1. Diagrama de flujo de CP2: por documentar.

Figura 2. Flujo del programa de telemetría de CP2 (diagrama pendiente).

III. Development

The purpose of this section is to answer both the checkpoint and report questions of the lab, as well as discuss any relevant results obtained during the experiences.

III-A. CHECKPOINT 1: Previous Installations

1. **Arduino Installation and ESP32 Setup**
   First, we downloaded and installed the Arduino IDE for our computer's operating system, which in this case was Windows 11, as shown in Figure 3.



Figura 3. Arduino IDE installation for Windows 11.

   Once installed and opened, we needed to add support for the ESP32 board. To do this, we went to File >Preferences. There, we added the URL https://dl.espressif.com/dl/package_esp32_index.json in the "Additional Boards Manager URLs" field and clicked "OK", as shown in Figure 4.
   Then, we needed to install the board support package for the ESP32. To do this, we went to Tools >Board >Boards Manager. There, we searched for "esp32" and installed the package provided by Espressif Systems, as shown in Figure 5.
   To finish the initial setup, we went to Select Board >Select Other Board and Port. There, we searched for "NodeMCU-32S" in the board selection section and selected "COM5 Serial Port" in the ports section, as shown in Figure 6.
2. **Test Code Upload**
   We tested a very simple code and actually ran into a small problem during the upload process. As shown in

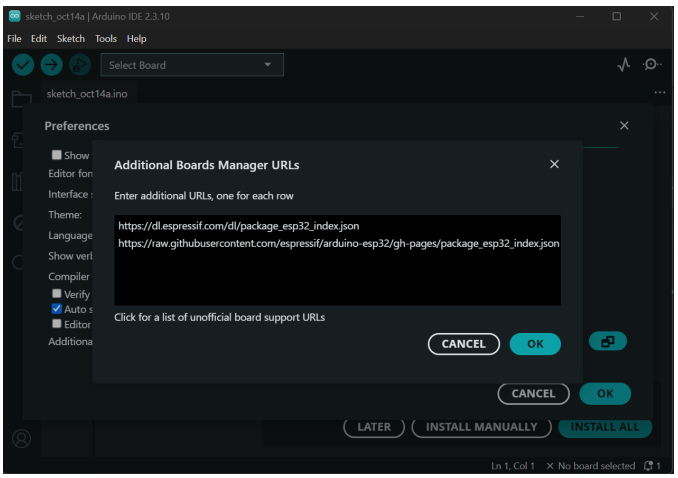


Figura 4. Arduino IDE preferences with the ESP32 board manager URL added.

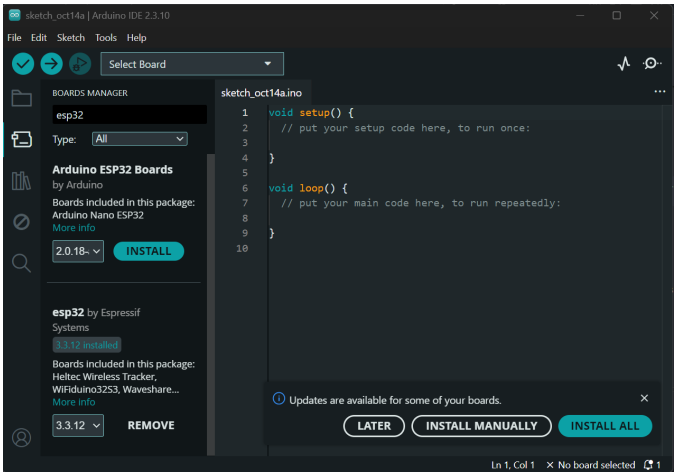


Figura 5. Installation of the ESP32 board support package by Espressif Systems.

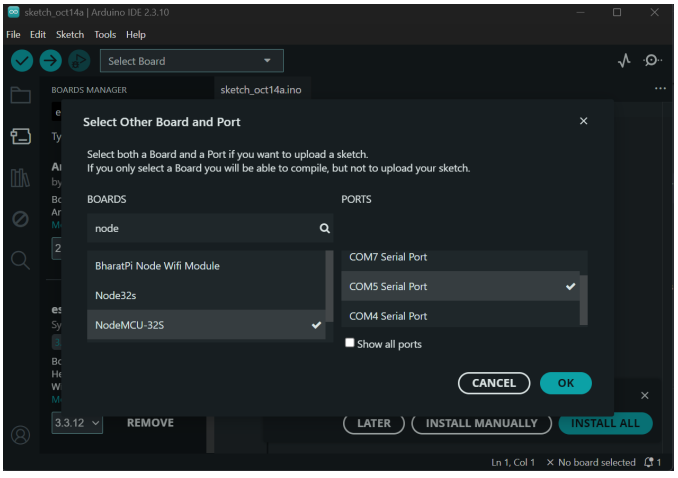


Figura 6. Initial selection of the NodeMCU-32S board and COM5 serial port in Arduino IDE.

Figure 7, there was an issue with the selected serial port. We solved this by changing “COM5 Serial Port” to “COM3 Serial Port (USB)”, which corresponded to the connected ESP32, as shown in Figure 8. After fixing this issue, we successfully uploaded and ran the test code on the ESP32, as shown in Figures 9 and 10.

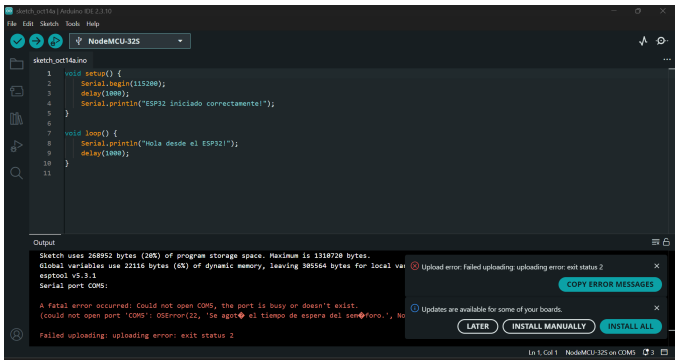


Figura 7. Error encountered during the initial code upload due to the incorrect serial port selection.

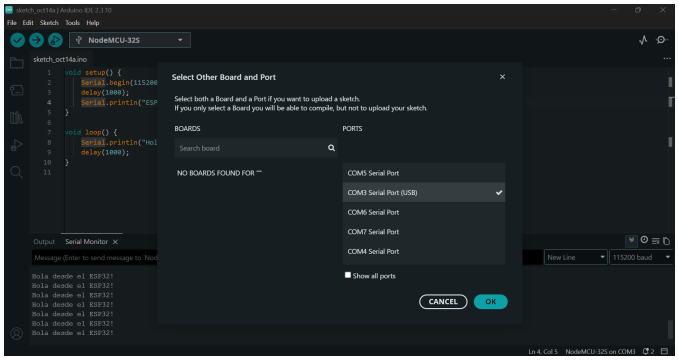


Figura 8. Correction of the serial port selection from COM5 to COM3 (USB).

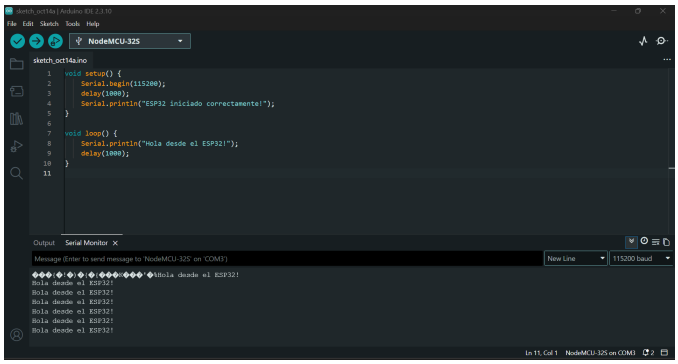


Figura 9. Successful compilation and upload of the test code to the ESP32.

3. **Mosquitto Installation**
First, we downloaded the Mosquitto executable installer for Windows, as shown in Figure 11. As instructed, Mosquitto was installed outside the Program Files directory. We also needed to add its installation path to the system environment variables in order to execute Mosquitto commands directly from the terminal. We

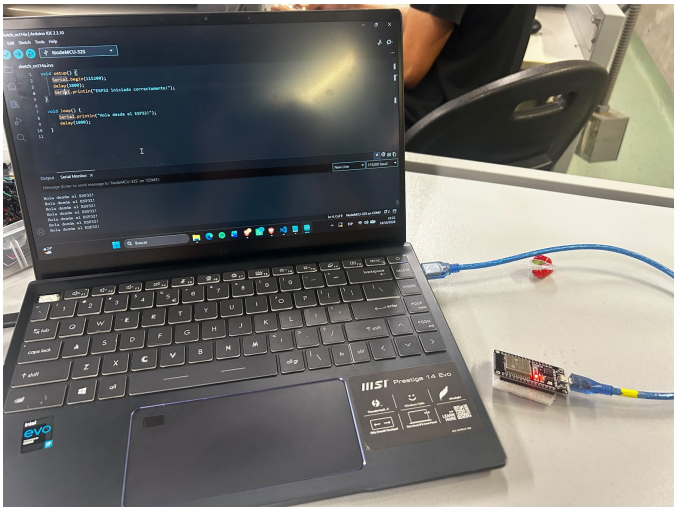


Figura 10. Arduino IDE Serial Monitor displaying the messages generated by the ESP32 test code.

then confirmed that Mosquitto could be executed successfully, as shown in Figure 12.

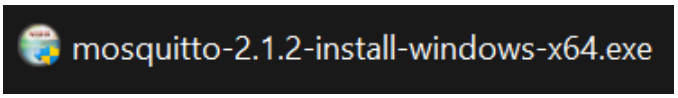


Figura 11. Download of the Mosquitto installer for Windows from its official website.

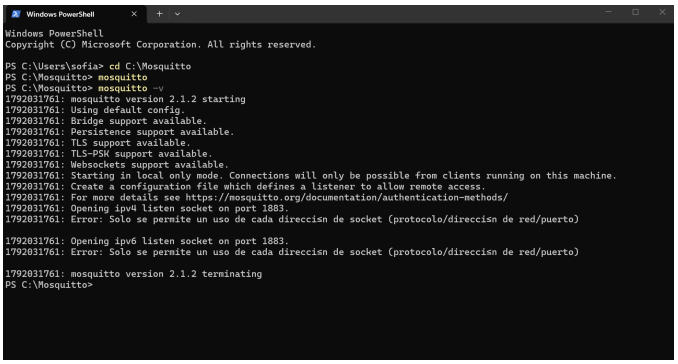


Figura 12. Verification of the Mosquitto installation through the Windows terminal.

Then, we modified the Mosquitto configuration file, as shown in Figure 13, and successfully ran the broker using the updated configuration, as shown in Figure 14. Finally, we tested MQTT communication by subscribing to the topic “test/topic” and publishing a test message to the same topic. As shown in Figure 16, we published a message, and as shown in Figure 15, we were able to receive it successfully through the subscribed client.

III-B. CHECKPOINT 2: Basic Implementation with Reading

IV. Conclusion

Conclusiones y recomendaciones experimentales: por documentar.

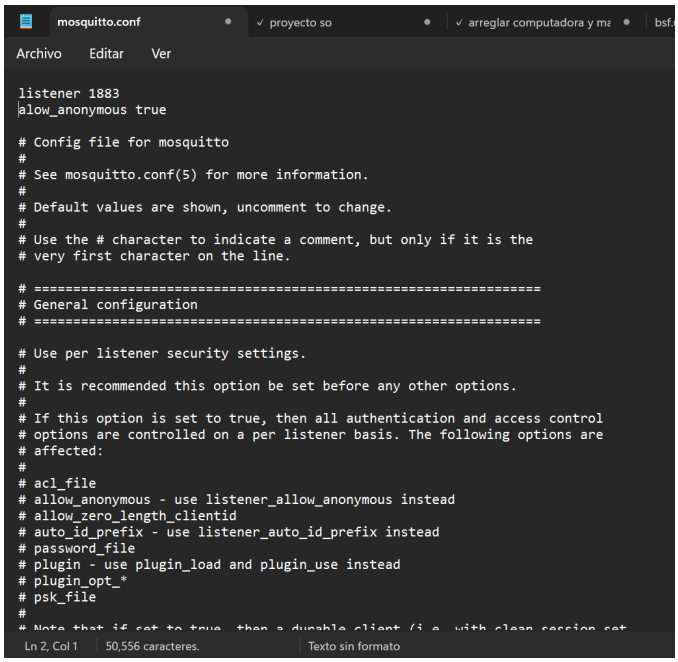


Figura 13. Modification of the Mosquitto configuration file to enable connections on port 1883 and allow anonymous clients.

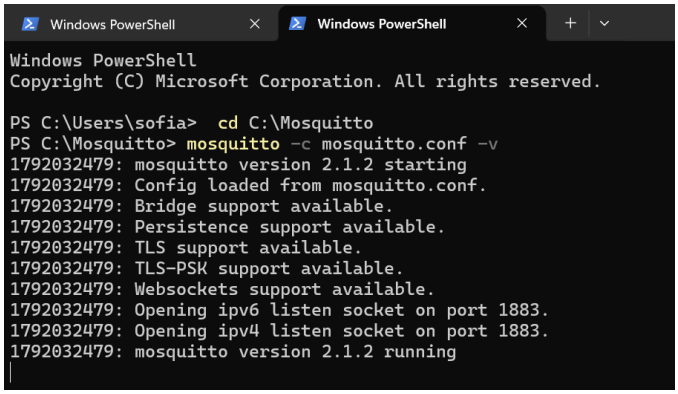


Figura 14. Successful execution of the Mosquitto MQTT broker using the updated configuration file.

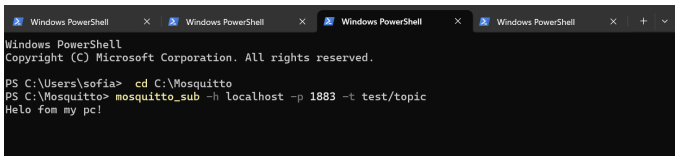


Figura 15. MQTT subscriber receiving the test message through the Mosquitto broker.

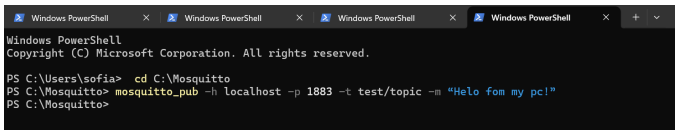


Figura 16. Publication of a test message to the MQTT topic using the Mosquitto command-line client.

REFERENCIAS

[1] *CS5055: LAB 3: MQTT & ESP32, Sem 09–Sem 10*, Universidad de Ingeniería y Tecnología, 2026, guía del curso, Sem10_LAB3_MQTT.pdf, pp. 1–11. Acceso institucional; consultada el 14 de octubre de 2026. [Online]. Available: https://utec.instructure.com/courses/24674/files/4935950

[2] *MQTT Version 3.1.1*, OASIS, Oct. 2014, oASIS Standard, 29 de octubre de 2014. Consultado el 14 de octubre de 2026. [Online]. Available: https://docs.oasis-open.org/mqtt/mqtt/v3.1.1/os/mqtt-v3.1.1-os.html

[3] *Arduino-ESP32: ADC API*, Espressif Systems, documentación oficial, funciones analogRead y analogReadMilliVolts. Consultada el 14 de octubre de 2026. [Online]. Available: https://docs.espressif.com/projects/arduino-esp32/en/latest/api/adc.html

[4] N. O'Leary, "PubSubClient, versión 2.8: código fuente," 2020, pubSubClient.h y PubSubClient.cpp, MQTT_VERSION, publish y loop. Consultados el 14 de octubre de 2026. [Online]. Available: https://github.com/knolleary/pubsubclient/tree/v2.8/src

[5] U. Hunkeler, H. L. Truong, and A. Stanford-Clark, "MQTT-S — A publish/subscribe protocol for Wireless Sensor Networks," in *2008 3rd International Conference on Communication Systems Software and Middleware and Workshops (COMSWARE)*, 2008, resumen disponible en IBM Research. Consultado el 14 de octubre de 2026. [Online]. Available: https://research.ibm.com/publications/mqtt-s-a-publishsubscribe-protocol-for-wireless-sensor-networks